



College Exam Timetable & Room Management System

Project Technical Specification & Viva Presentation Manual

System Domain: Educational Examination Management

Architecture: Client-Server Fullstack Architecture

Technology Stack: Node.js, Express.js, MongoDB, Mongoose, HTML5, CSS3, ES6 JavaScript

Security / Auth: JSON Web Tokens (JWT), BcryptJS Password Hashing

GitHub Repository: <https://github.com/samikshamalange2005-byte/student-exam-portal>

1. Project Overview & Executive Summary

The **College Examination Timetable & Room Management System** is an end-to-end web portal designed to streamline university examination scheduling and eliminate physical notice-board bottlenecks.

The application provides two completely separated, role-based workflows:

- **Admin Portal (Exam Cell Faculty):** Provides faculty administrators with tools to create exam schedules, define examination halls with seating capacities, enforce room capacity restrictions, and prevent overlapping room booking clashes.
- **Student Portal:** Allows registered students to sign in and immediately access an uncluttered, personalized exam timetable that dynamically displays *only* the examinations applicable to their enrolled academic year and class section.

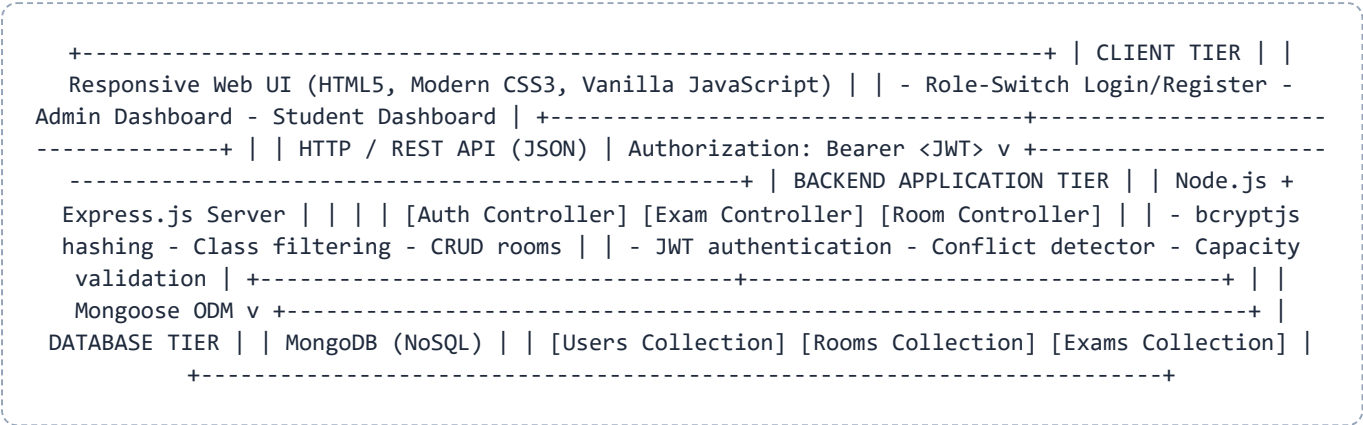
 30-Second Elevator Pitch for Viva / Interview:

"This project is a role-based full-stack web application that solves common examination logistics issues in colleges: it eliminates student confusion by strictly filtering exam timetables to a student's enrolled year and section, validates that assigned students do not exceed room seating limits, and mathematically prevents double-booking of exam venues at overlapping times."

2. Real-World Problems Solved

Traditional College Problem	Consequences	System Solution
Master Notice Board Clutter	Students comb through huge 50-page PDFs or crowded notice boards, causing missed papers or confusion.	Class-Specific Filtering: Students only see examinations for their specific Year and Section.
Room Double-Booking Clashes	Two teachers schedule different exams in the same hall simultaneously, resulting in last-minute venue chaos.	Mathematical Clash Prevention: System checks time intervals and rejects overlapping bookings for the same hall.
Hall Overcrowding	Exams are assigned to small classrooms that cannot seat all enrolled students.	Capacity Limit Validation: Server rejects exam scheduling if student count exceeds room capacity.

3. System Architecture & Tech Stack



Component Breakdown:

- **Frontend (Vanilla HTML5, CSS3, ES6 JS):** Lightweight, zero compilation build steps, fast loading, featuring custom CSS variables, responsive grids, and media print styling for student exam passes.
- **Backend (Node.js & Express.js):** RESTful API server with asynchronous non-blocking I/O handling authentication, room management, and clash detection.
- **Database (MongoDB with Mongoose):** High-performance NoSQL document store with strict Mongoose schema validation.
- **Authentication (JWT & BcryptJS):** Passwords securely hashed with 10 salt rounds; stateless JWT tokens maintain session security without server-side memory overhead.

4. Database Schema & Data Models

1. User Schema (`models/User.js`)

Field	Type	Description
<code>name</code>	String	Full Name of Student or Administrator
<code>email</code>	String	Unique login email
<code>password</code>	String	Cryptographic hash produced by BcryptJS
<code>role</code>	String	<code>'admin'</code> or <code>'student'</code>
<code>rollNo</code>	String	Student ID / Roll number (e.g. <code>24CS001</code>)
<code>department</code>	String	e.g. Computer Science & Engineering
<code>year</code>	String	Enrolled Year (e.g. <code>'1st Year'</code> , <code>'2nd Year'</code> , etc.)
<code>section</code>	String	Class Section (e.g. <code>'A'</code> , <code>'B'</code> , <code>'C'</code>)

2. Room Schema (`models/Room.js`)

Field	Type	Description
<code>name</code>	String	Unique room/hall name (e.g. <code>Hall 101</code> , <code>Auditorium</code>)
<code>capacity</code>	Number	Maximum seating capacity (must be ≥ 1)

3. Exam Schema (`models/Exam.js`)

Field	Type	Description
<code>subjectName</code>	String	Course name (e.g. <i>Data Structures & Algorithms</i>)
<code>subjectCode</code>	String	Course code (e.g. <code>CS201</code>)
<code>examDate</code>	String	Date formatted as <code>YYYY-MM-DD</code>
<code>startTime</code> / <code>endTime</code>	String	Exam session times (e.g. <code>10:00 AM</code> to <code>01:00 PM</code>)
<code>year</code> / <code>section</code>	String	Target class receiving the timetable
<code>room</code> / <code>roomId</code>	String / ObjectId	Assigned examination venue
<code>roomCapacity</code>	Number	Seat limit of the assigned room
<code>studentCount</code>	Number	Number of students assigned for the exam

5. Core Algorithms & Logic Explained

A. Strict Class-Specific Timetable Filtering

When a student logs in and accesses `GET /api/exams`, the backend verifies their JWT token, extracting their enrolled `year` and `section`. The query strictly restricts the returned documents:

```
if (req.user.role === 'student') {  
  query.year = req.user.year;  
  query.$or = [  
    { section: req.user.section },  
    { section: 'All' }  
  ];  
}
```

Guaranteed Result: A 2nd-year Section A student will never see 3rd-year Section B exams, preventing cross-class timetable confusion.

B. Room Seating Capacity Limit Check

Before an exam is created, the system checks the assigned student count against the room's maximum seating capacity:

```
if (assignedStudents > targetRoom.capacity) {  
  return res.status(400).json({  
    message: `Room capacity exceeded! Room "${targetRoom.name}" has a capacity of ${targetRoom.capacity}.  
  });  
}
```

C. Time-Interval Overlap Conflict Formula

To verify whether two exams scheduled in the same hall clash, start and end times are parsed into **total minutes from midnight**:

- `10:00 AM` = $(10 \times 60) + 0 = \mathbf{600 \text{ minutes}}$
- `01:00 PM` = $(13 \times 60) + 0 = \mathbf{780 \text{ minutes}}$

Given two time intervals $[S_1, E_1]$ and $[S_2, E_2]$, they **overlap if and only if**:

$$\max(\text{Start}_1, \text{Start}_2) < \min(\text{End}_1, \text{End}_2)$$

If this condition evaluates to `true` for any exam already booked in the same hall on that date, the request is rejected with HTTP 400 and a descriptive conflict alert. Consecutive sessions (e.g. 10:00 AM–01:00 PM and 02:00 PM–05:00 PM) do not overlap and are accepted cleanly.

6. REST API Specification

Method	Path	Auth Role	Purpose
POST	/api/auth/register	Public	Registers student or admin (with admin key)
POST	/api/auth/login	Public	Authenticates credentials, returns JWT token
GET	/api/auth/me	Authenticated	Retrieves active user profile
GET	/api/rooms	Authenticated	Lists all rooms with capacities
POST	/api/rooms	Admin	Creates new room with capacity
DELETE	/api/rooms/:id	Admin	Deletes room (safely guarded against active exams)
GET	/api/exams	Authenticated	Fetches class-filtered exams for student, all for admin
POST	/api/exams	Admin	Schedules exam with capacity & clash checks
DELETE	/api/exams/:id	Admin	Cancels / removes an exam schedule
GET	/api/exams/stats	Admin	Returns KPI dashboard summary counts

7. Step-by-Step Live Viva / Demo Script

1. Home Page Walkthrough (`http://localhost:5000`):

Highlight the dual-portal landing cards for *Admin Portal* and *Student Portal*.

2. Admin Dashboard Demo (`admin@college.edu` / `admin123`):

- Show KPI overview counters (Total Scheduled, Students, Rooms, Upcoming).
- Open *Manage Rooms*: Add `Physics Lab 1` with capacity `40`. Show it appears immediately.
- **Demonstrate Capacity Enforcement**: Try to schedule an exam in `Physics Lab 1` with 50 students. Point out the clear rejection alert.
- **Demonstrate Clash Prevention**: Schedule an exam in `Hall 101` from 10:00 AM to 01:00 PM. Attempt to schedule another in `Hall 101` from 11:30 AM to 02:30 PM. Show the clash alert. Change time to 02:00 PM – 05:00 PM and show it succeed.

3. Student View Demo (`alice@college.edu` / `student123`):

- Log in as Alice (2nd Year, Section A).
- Point out the notice banner confirming class-specific filtering.
- Show that Alice sees *only* 2nd Year Section A papers with assigned rooms, capacities, and timing badges.
- Click **Print Timetable** to demonstrate the printable admit slip.

4. Cross-Check with Second Student (`bob@college.edu` / `student123`):

Sign in as Bob (3rd Year, Section B) to prove that Bob receives a completely different timetable tailored to 3rd-year classes.

8. Common Viva Questions & Model Answers

Q1: Why use JWT instead of standard cookie sessions?

JWT tokens are stateless and self-contained. The server does not maintain session memory, allowing the application to scale horizontally across multiple instances without sticky sessions or centralized Redis session stores.

Q2: How are passwords secured in the database?

Passwords are never stored in plaintext. They are encrypted using `bcryptjs` with 10 salt rounds, utilizing one-way salted hashing to resist rainbow table and dictionary attacks.

Q3: How does the clash detection algorithm work with 12-hour AM/PM times?

The backend parses 12-hour AM/PM strings into total minutes from midnight (0 to 1439). It then evaluates whether $\max(S_1, S_2) < \min(E_1, E_2)$.

Q4: Can an admin delete a room that is currently in use?

No. The `DELETE /api/rooms/:id` route queries active scheduled exams assigned to that room and blocks deletion if active schedules exist, preserving relational data integrity.

9. Quick Run Commands

```
# Start the Node.js server
npm start

# Run room capacity & clash prevention automated test suite
node test-rooms-clash.js

# Run authentication and class filtering test suite
node test-api.js
```